



Centro de
Informática
UFPE

TiCoder vs. CodeT: Reprodução e Análise Comparativa

Avaliação de Precisão via Benchmarks.

Gustavo Santiago<gssm> João Teles<jant> Matheus Borges<mbf3>
Mikael Cavalcanti<mcs11> Pedro Domingues<prds> Rafael Leite<rlo3>

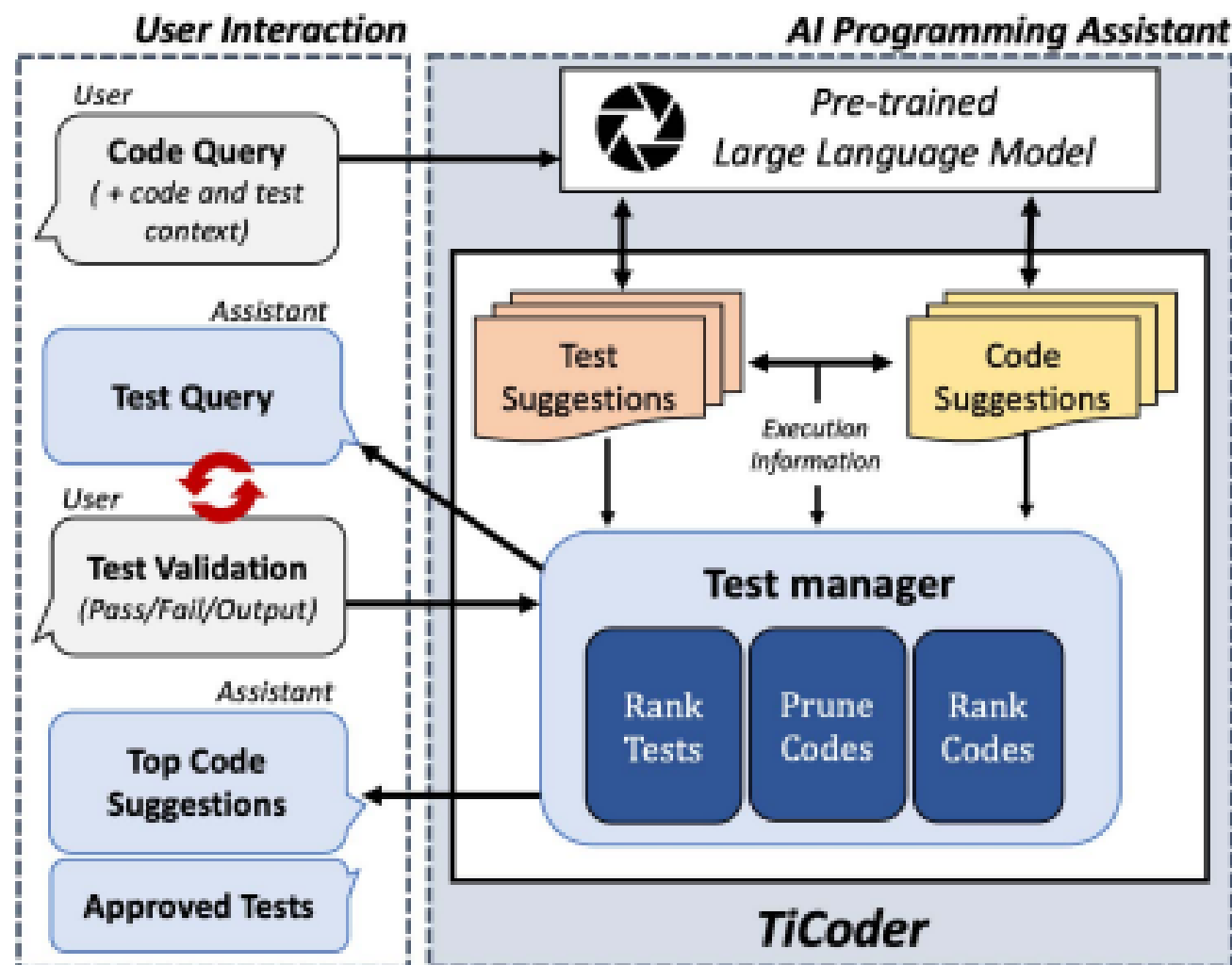
Motivação

- Objetivo do Ti-Coder: Aumentar a precisão na seleção de códigos corretos.
- É possível alcançar a mesma precisão com ferramentas totalmente automatizadas (sem intervenção humana)?
- Artigo do TiCoder **não apresenta** comparação dos seus resultados com o seu antecessor, o CodeT, citado em trabalhos anteriores, que faz justamente isso
- Em seu artigo original, CodeT é testado contra os mesmos benchmarks, com precisão marginalmente inferior, porém, cada tarefa usava 100 gerações!

Introdução

TiCoder VS CodeT

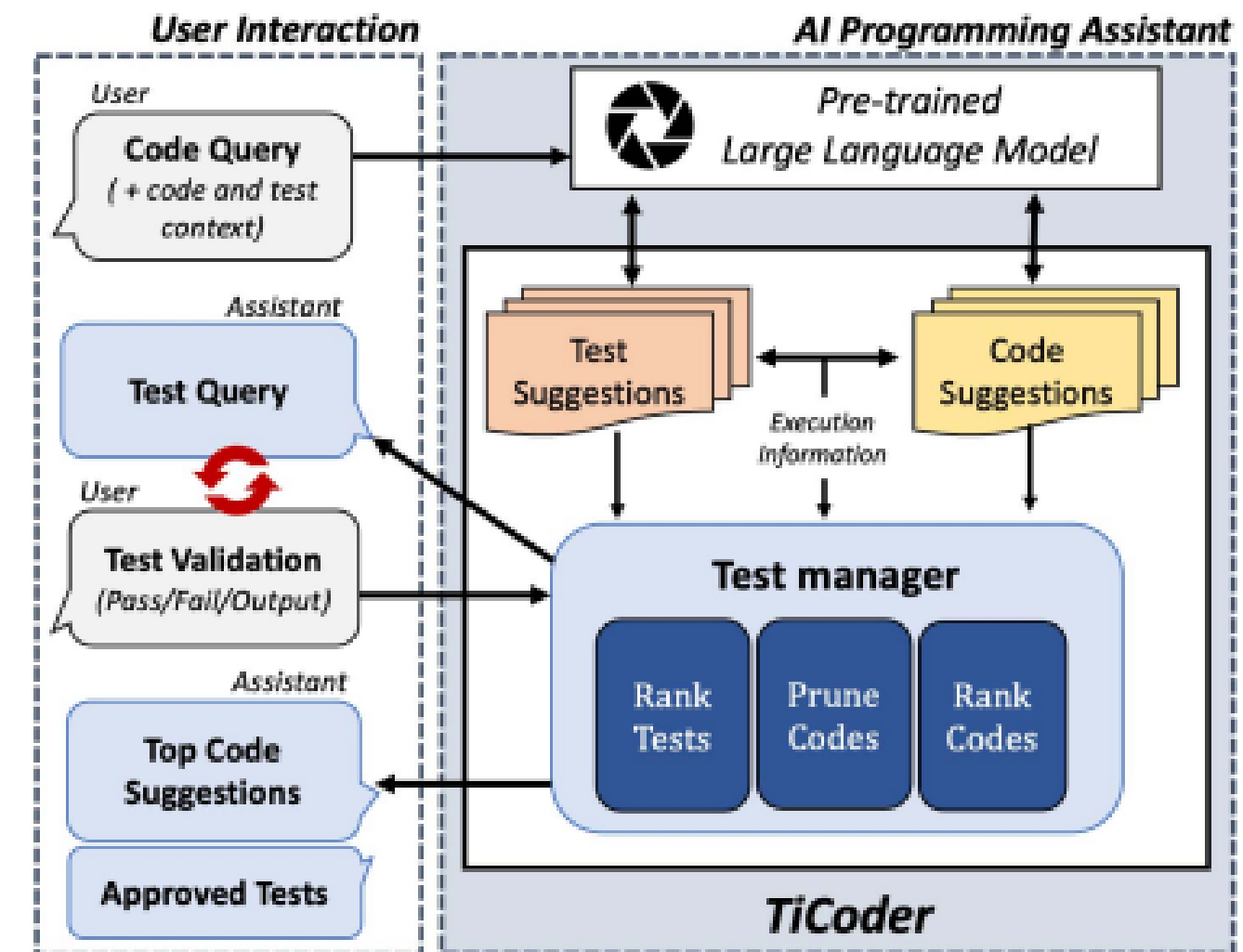
TiCoder



$$s_{discr}(t) = \begin{cases} 0 & \text{if } \max(|G_t^+|, |G_t^-|) \text{ is } 0 \\ \frac{\min(|G_t^+|, |G_t^-|)}{\max(|G_t^+|, |G_t^-|)} & \text{otherwise} \end{cases}$$

TiCoder

- Pass Fail
- Output
- Interativo
- Com oráculo



Interação com TiCoder

You want to generate a python function using the following prompt:

Write a python function to find the smallest missing number from a sorted list of integers, starting from 0.

Before generating code suggestions, AI Assistant #2 wants to ask you a few questions:

Should the following assertion pass or fail?

```
assert find_First_Missing([1,2,4,6]) == 0
```

PASS

FAIL

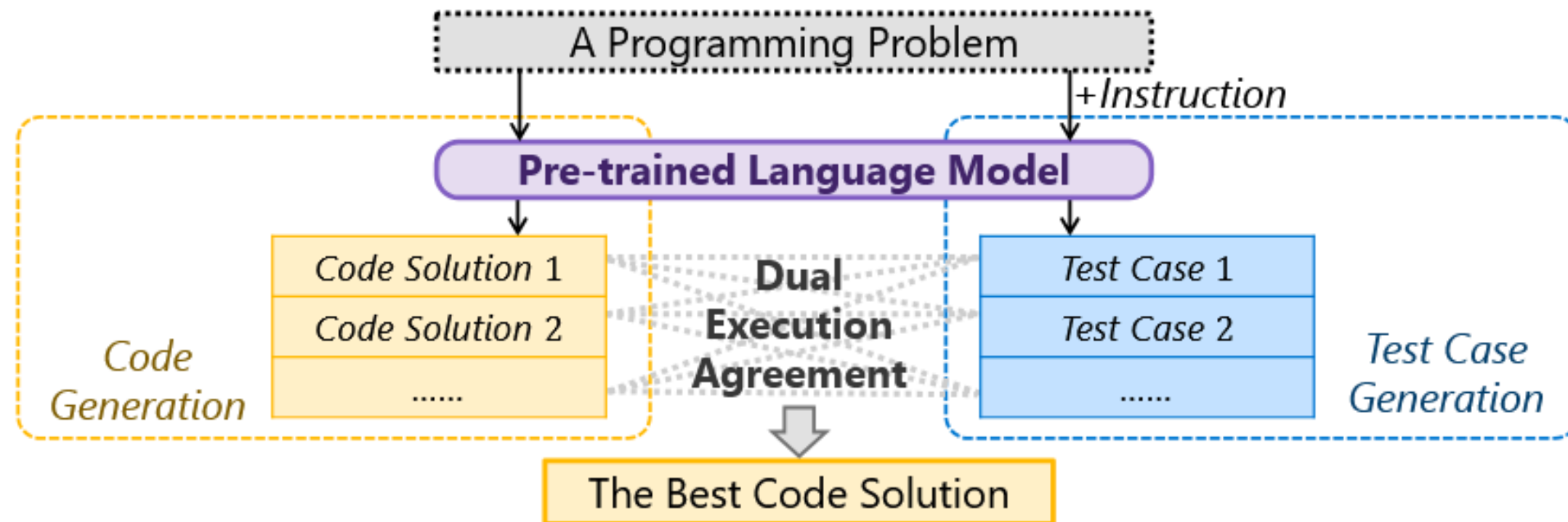
You want to generate a python function using the following prompt:

Write a function to find the maximum product formed by multiplying numbers of an increasing contiguous subsequence of that array. The sequence may include negative numbers.

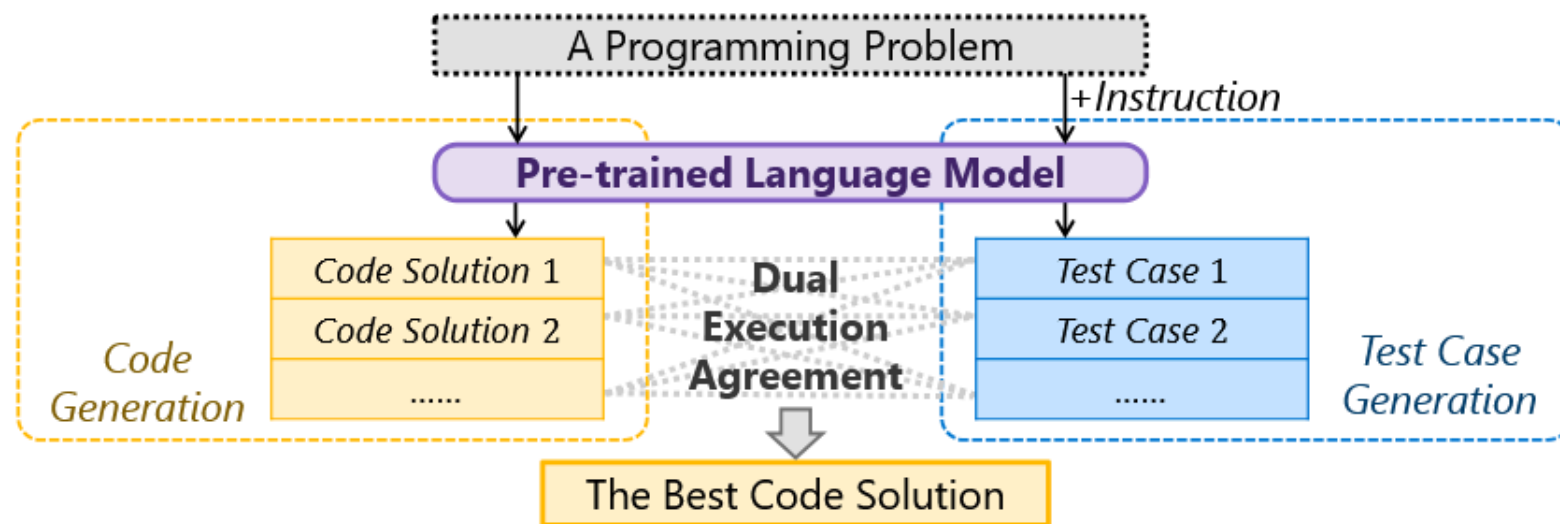
Before generating code suggestions, AI Assistant #3 wants to ask you a few questions:

```
assert max_product([3, 4, 2]) == ?
```

CodeT



Dual Execution Agreement



- **Geração Dupla:** O modelo (LLM) recebe o problema e gera código e testes simultaneamente
- Executa **todos** os **códigos** gerados **contra todos** os **testes** gerados.
- **Busca consenso:** Grupo de soluções que passou no maior número de testes e produziu a mesma saída
- **Critério de seleção:**
 - **Robustez:** passar nos testes
 - **Consistencia:** concordar com outras soluções

Por que funciona sem humanos?

- **Qualidade via Quantidade**
- **Redução de Ruído**
- **Seleção Automática**

Benchmarks: TiCoder

Dataset	Model	Baseline pass@k		TiCODER-PASSFAIL pass@1@m					TiCODER-OUTPUT pass@1@m				
		1	100	1	2	3	4	5	1	2	3	4	5
MBPP	text-davinci-003	49.16	86.88	68.04	75.26	77.33	77.88	78.08	77.00	82.20	83.38	83.59	83.75
	code-davinci-002	48.25	89.75	68.42	76.21	79.37	81.18	81.97	76.97	85.51	87.10	87.56	87.71
	CodeGen-6B	14.85	69.55	28.62	37.91	45.18	49.97	53.67	39.64	54.01	62.34	65.30	66.56
	CodeGen2.5-7B	28.32	84.74	50.27	59.70	65.02	67.84	69.58	62.78	73.84	78.10	79.52	80.42
	GPT-3.5-turbo	61.91	84.77	75.12	77.31	78.21	79.04	79.03	78.24	80.86	81.06	81.76	81.78
	GPT-4-turbo	69.80	86.88	80.71	81.90	82.38	83.20	83.12	83.91	84.63	84.97	85.65	85.65
	GPT-4-32k	67.13	87.35	81.56	82.62	82.97	83.11	83.70	84.78	85.28	85.31	85.40	85.79
HumanEval	text-davinci-003	44.13	87.80	60.70	67.54	71.41	72.18	72.81	65.02	75.48	78.04	79.34	80.18
	code-davinci-002	30.49	91.49	51.66	62.65	70.30	73.11	74.37	57.25	74.5	81.99	83.70	84.50
	CodeGen-6B	11.41	43.55	15.32	19.29	24.64	28.11	29.56	26.34	32.81	39.08	44.23	48.12
	CodeGen2.5-7B	21.39	76.21	32.82	41.03	46.51	49.47	52.33	35.86	51.60	61.54	65.02	68.25
	GPT-3.5-turbo	59.45	89.02	73.16	76.48	77.01	77.75	79.22	73.44	76.60	78.45	78.45	79.51
	GPT-4-turbo	62.62	89.63	78.36	80.42	80.92	80.84	81.34	82.22	83.17	83.42	83.62	83.84
	GPT-4-32k	60.72	89.02	76.10	78.43	79.07	79.48	79.49	81.37	82.46	82.49	82.49	82.54

TABLE IV: Model baseline and TiCoder results for two python datasets: MBPP and HumanEval. User interaction results are simulated for up to m=5 test evaluation interactions. We highlight, in blue, the highest accuracy in each column.

Benchmarks: CodeT

Methods	Baseline			AlphaCode-C			CODET		
k	1	10	100	1	2	10	1	2	10
HumanEval									
code-cushman-001	33.5	54.3	77.4	39.6	46.4	63.8	44.5 11.0	50.1	65.7 11.4
code-davinci-001	39.0	60.6	84.1	41.6	50.7	75.6	50.2 11.2	58.9	75.8 15.2
code-davinci-002	47.0	74.9	92.1	55.1	64.1	84.4	65.8 18.8	75.1	86.6 11.7
INCODER-6B	16.4 15.2	28.3 27.8	47.5 47.0	17.7	23.8	34.8	20.6 4.2	27.6	37.1 8.8
CODEGEN-MONO-16B	29.7 29.3	50.3 49.9	73.7 75.0	27.3	38.5	64.4	36.7 7.0	44.7	59.3 9.0
MBPP									
code-cushman-001	45.9	66.9	79.9	51.5	59.0	73.3	55.4 9.5	61.7	72.7 5.8
code-davinci-001	51.8	72.8	84.1	56.2	64.7	78.8	61.9 10.1	69.1	79.3 6.5
code-davinci-002	58.1	76.7	84.5	62.0	70.7	79.9	67.7 9.6	74.6	81.5 4.8
INCODER-6B	21.3 19.4	46.5	66.2	26.7	35.3	56.2	34.4 13.1	43.9	58.2 11.7
CODEGEN-MONO-16B	42.4	65.8	79.1	41.0	55.9	73.6	49.5 7.1	56.6	68.5 2.7

Table 2: Pass@ k (%) on the HumanEval and MBPP benchmarks. AlphaCode-C is our replication of the clustering method in Li et al. (2022b). The numbers in red indicate the absolute improvements of CODET over baseline on pass@1 and pass@10. We also list the baseline results from Fried et al. (2022a) and Nijkamp et al. (2022) for reference in gray, where the settings of context are not exactly the same as ours. For CODET, temperature is set to 0.8 and sampling number is set to 100. We do not show CODET pass@100, since it is the same as the baseline pass@100.

Reprodução e Comparação

TiCoder VS CodeT

Metodologia

Proposta: Comparar os dois frameworks sob as mesmas condições, para controlar fatores externos

- **Mesmos códigos**
- **Mesmos testes**
- **Mesmo ambiente**

Experimento: 21 exemplos do MBPP amostrados aleatoriamente

Prompts: usamos prompts baseados nos prompts do TiCoder para geração de códigos e testes

Implementação: Fizemos versões simplificadas de ambos os frameworks

Modelo usado: GPT-3.5-Turbo

TiCoder:

- **Ranking:** Distinguishing power
- **Interação:** Oráculo
- **Modos:** Ambos Pass/Fail e Output

Resultados

Métricas	Resultados
Baseline (Pass@1)	70,33%
TiCoder (Pass-Fail)	~71%
Ticoder (Output)	72,25%
CodeT (Consensus)	75,83%

Hipóteses

Amostragem pequena

21 exemplos são insuficientes para convergir

TiCoder ‘cru’

Implementação específica dos autores era muito mais complexa do que o framework proposto, gerando resultados melhores



Muito
obrigado!



Centro de
Informática
UFPE